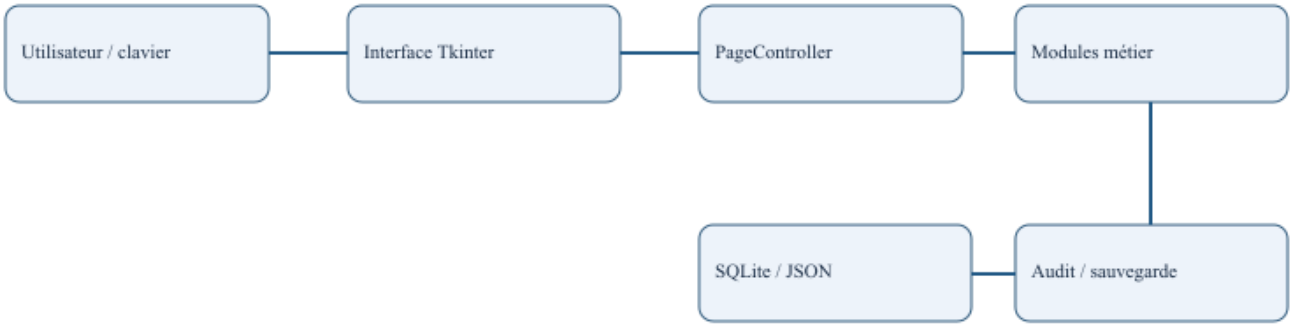
StorageScope v6.16 — Audit UI et fiabilité

Auteur : Abdelhalim Talbi



1. Objectif et périmètre

StorageScope est une console Windows-first de gouvernance infrastructure, architecture et stockage. La v6.16 traite un défaut de démarrage, la cohérence des dimensions DPI et une erreur potentiellement masquée dans le chargement asynchrone. Les tests sont exécutés sous Linux, pas sur un poste Windows.

2. Audit du code et des composants

Inventaire automatique : 261 modules Python et 381 relations d’import interne. Les composants étudiés comprennent la fenêtre principale Tkinter, PageController, les pages Gouvernance/Campagnes/Diagnostic, le chargeur asynchrone et les registres SQLite. Le rapport JSON détaille les importations manquantes sélectionnées et les clauses except sans type.

3. Défaut bloquant détecté

Avant correction, le démarrage importait windows_accessibility_v611, module absent des sources de v6.15. La v6.16 utilise windows_accessibility_v616, présent dans le ZIP. Le test de non-régression vérifie explicitement cette dépendance.

4. Résolution et mise à l’échelle

Les seuils de mode compact sont désormais évalués en pixels logiques, en divisant les dimensions physiques par le facteur DPI obtenu par Tk. À 1920×1080 et 200 %, la zone logique vaut 960×540 et le mode compact est activé. À 100 %, le mode normal est retenu.

5. Cycle de vie asynchrone

Le chargeur de portfolio ouvre une connexion SQLite en lecture seule, active query_only, ouvre une transaction de lecture cohérente, produit le résultat, puis valide. En cas d’échec, une annulation est tentée sans masquer l’erreur d’origine. La fermeture est protégée contre les erreurs sqlite3.Error.

6. Architecture et flux

Utilisateur → raccourcis clavier / recherche → interface Tkinter → PageController → modules métier → SQLite et préférences JSON → journaux / sauvegardes. Les accès au stockage doivent rester hors du thread graphique pour les opérations coûteuses.

7. Parcours d’utilisation

Accueil : indicateurs et accès rapides. Gouvernance : consulter et filtrer, sélectionner une décision, examiner sa chronologie. Campagnes : comparer les historiques. Diagnostic : vérifier les bases. Maintenance : réaliser une sauvegarde et vérifier son manifeste. Les actions sensibles restent soumises à validation humaine.

8. Procédure de compilation Windows

Extraire le ZIP ; exécuter Construire-StorageScope.cmd dans un terminal Windows disposant de Python, PowerShell et des outils prévus par les scripts de compilation. Contrôler les journaux et le répertoire installer. Le précontrôle Linux n’est pas une compilation Windows.

9. Qualification et scénarios de recette

Tester Windows 10/11 en VM, affichage 1024×768, 1280×720, 1920×1080, DPI 100/150/200 %, Ctrl+F, Ctrl+1/2/3, F5, Escape, focus, ordre de tabulation, navigation avec et sans base, reprise après erreur SQLite, sauvegarde/restauration et désinstallation. Documenter la version OS et joindre des captures authentiques.

10. Captures et preuves

Aucune capture réelle de l’application v6.16 sous Windows n’a été obtenue. Le schéma d’architecture fourni est une illustration technique créée pour cette documentation. Les exemples de console ci-dessous sont des SIMULATIONS, pas des captures de commandes exécutées.

11. Exemple CLI — SIMULATION

Commande fictive de qualification Windows : python storagescope_windows.py --version

Résultat attendu (SIMULATION) : 6.16

Commande fictive : python verify-windows-build.py

Résultat attendu (SIMULATION) : PASS Windows build preflight 6.16

12. Exemple API — SIMULATION

StorageScope v6.16 n’expose pas de service HTTP obligatoire dans son interface locale. Exemple illustratif NON IMPLÉMENTÉ : GET /api/v1/health → {"status":"ok","version":"6.16"}. Ne pas interpréter cet exemple comme une route réellement disponible.

13. Risques résiduels et priorités

Le démarrage et le rendu natifs Windows n’ont pas été observés. La densité de commandes dans certains écrans, le débordement des barres de filtres et les temps de chargement de gros registres nécessitent une recette visuelle. Les imports dynamiques ne sont pas tous couverts par l’inventaire statique. Prochaines étapes : tests automatisés Windows en VM et mesure de performances.